

Cours Développement Mobile

Fondements du Développement Android : Manifest, Activités et Cycle de Vie

Salem Issa & Med Lemine

Institut Supérieur du Numérique (SupNum)

2024-2025

À la fin de ce cours, vous serez capables de :

- Définir le rôle du fichier Manifest
- Expliquer le concept d'activité
- Analyser le cycle de vie d'une activité
- Illustrer l'interaction entre Java et XML

Plan du cours

- 1 Fichier de configuration
- 2 Activité
- 3 Cycle de Vie d'une Activité Android
- 4 Cycle de Vie d'une Activité Android
- 5 Cycle de vie d'une activité

Qu'est-ce que AndroidManifest.xml ?

Le fichier **AndroidManifest.xml** est un fichier fondamental dans une application Android. Il joue un rôle crucial dans la définition de la structure et du comportement de l'application.

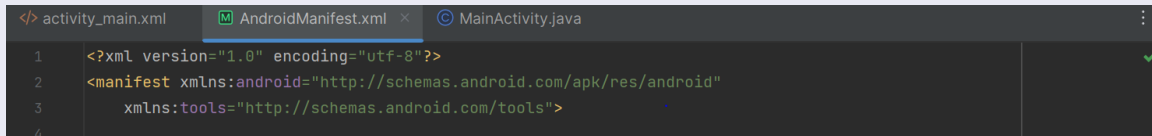
Rôles principaux

- Déclaration des composants de l'application (Activity, Service, BroadcastReceiver, ContentProvider).
- Gestion des permissions requises (INTERNET,
- Configuration globale de l'application (thème, orientation de l'écran, paramètres matériels).
- Spécification du point d'entrée de l'application avec l'Activity principale.

Le fichier **AndroidManifest.xml** est structuré avec plusieurs balises essentielles :

Balise <manifest>

Définit le package de l'application (package="com.example.myapp").



```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
```

Description :

- **xmlns:android** : Définit l'espace de noms XML.
- **package** : Identifiant unique de l'application.

Balis <application>

```
<application
    android:allowBackup="true"
    android:dataExtractionRules="@xml/data_extraction_rules"
    android:fullBackupContent="@xml/backup_rules"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:theme="@style/Theme.MyApplication"
    tools:targetApi="31">
```

- android:roundIcon : Icônes de l'application.
- android:label : Nom affiché de l'application.
- android:theme : Thème de l'application.
- android:dataExtractionRules et android:fullBackupContent : Règles de sauvegarde

Balise <activity>

```
<activity
    android:name=".MainActivity"
    android:exported="true">

    <intent-filter>
        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
```

- Déclare une activité (écran) de l'application.
- Exemple : `android:name=".MainActivity"` définit la classe de l'activité principale.
- `android:exported="true"` indique que l'activité peut être lancée depuis l'extérieur de l'application.

Interconnection entre Layout et Java

En Android, l'interface utilisateur et la logique métier sont séparées :

- **Activity (Java)** : Gère la logique et les interactions.
- **XML (Layout)** : Définit l'interface utilisateur.
- **R.java** : Fait le lien entre les deux.

Qu'est-ce qu'une Activity ?

- Une **Activity** est un composant fondamental d'une application Android.
- Chaque écran d'une application correspond généralement à une Activity.
- Elle gère l'affichage et les interactions avec l'utilisateur.
- Exemple : Page d'accueil, écran de connexion.

Une Activity est composée de :

- Un **fichier Java** qui gère la logique.
- Un **fichier XML** qui définit l'interface utilisateur (UI).

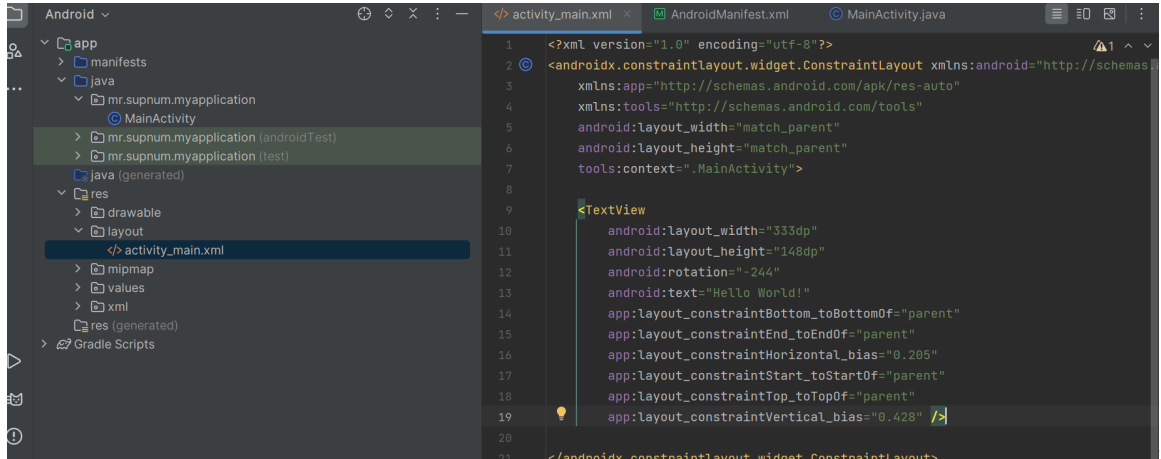
1. Partie Java : MainActivity.java

- Hérite de la classe AppCompatActivity.
- Redéfinit la méthode onCreate() qui initialise l'Activity.
- Utilise setContentView() pour charger le fichier XML.

```
1 package mr.supnum.myapplication;
2
3 import androidx.appcompat.app.AppCompatActivity;
4
5 import android.os.Bundle;
6
7 </> public class MainActivity extends AppCompatActivity {
8
9     @Override
10     protected void onCreate(Bundle savedInstanceState) {
11         super.onCreate(savedInstanceState);
12         setContentView(R.layout.activity_main);
13     }
14 }
```

2. Partie XML : activitymain.xml

- Définit l'interface utilisateur de l'Activity.
- Chaque élément a un id pour être utilisé en Java.



Comment Java et XML sont liés ?

L

a classe `R` en Android joue un rôle fondamental dans la liaison entre Java et XML. Elle permet aux fichiers Java d'accéder aux ressources définies en XML (layouts, chaînes de caractères, images, couleurs, etc.).

Activity utilise **R.layout** pour afficher l'UI.

R.id permet d'accéder aux éléments de l'UI.

R.java connecte le code Java aux ressources XML.

Définition de la classe R.java

- Android Studio génère automatiquement une classe spéciale appelée **R.java**.
- Cette classe en **lecture seule** contient des identifiants uniques pour toutes les ressources (layouts, strings, images, etc.).
- Elle est mise à jour automatiquement en fonction des fichiers XML présents dans le dossier res/.

```
1 package com.example.myapp;
2
3 public final class R {
4     public static final class layout {
5         public static final int activity_main = 0x7f0a0001;
6     }
7
8     public static final class id {
9         public static final int textView = 0x7f0b0002;
10    }
11
12    public static final class drawable {
13        public static final int logo = 0x7f0c0003;
14    }
15 }
```

Explication du code

- `setContentView(R.layout.activity_main);` Charge l'interface définie en `activity_main.xml`.
- `findViewById(R.id.hello);` l'élément `TextView` défini en XML.
- `findViewById(R.id.logoapp);` → Récupère l'image défini en XML.
- Ajout d'un événement `setText()` r modifier le texte .

```
1 package mr.supnum.myapplication;
2
3 import androidx.appcompat.app.AppCompatActivity;
4
5 import android.os.Bundle;
6 import android.widget.ImageView;
7 import android.widget.TextView ;
8
9 </> public class MainActivity extends AppCompatActivity {
10     @Override
11     protected void onCreate(Bundle savedInstanceState) {
12         super.onCreate(savedInstanceState);
13         setContentView(R.layout.activity_main);
14
15         TextView myTextView = findViewById(R.id.hello);
16         myTextView.setText("Bienvenue !");
17
18         // ImageView myImageView = findViewById(R.id.logoapp);
19         // myImageView.setImageResource(R.drawable.logo);
```

Classe R et Organisation des Ressources

La classe R est divisée en plusieurs sous-classes pour organiser les ressources d'une application Android.

Catégorie	Description	Exemple d'accès en Java
R.layout	Définit les fichiers XML des interfaces.	<code>setContentView(R.layout.activity_main);</code>
R.id	Identifiants des éléments UI.	<code>findViewById(R.id.button);</code>
R.string	Chaînes de caractères stockées.	<code>getString(R.string.app_name);</code>
R.drawable	Images et icônes.	<code>imageView.setImageResource(R.drawable.logo);</code>
R.color	Couleurs définies en XML.	<code>textView.setTextColor(getResources().getColor(R.color.black));</code>
R.style	Styles et thèmes.	<code>setTheme(R.style.Theme_MyApp);</code>
R.raw	Ressources brutes (ex. fichiers audio, JSON).	<code>MediaPlayer.create(this, R.raw.musique);</code>

Étapes du cycle de vie

- **onCreate()** : Initialisation de l'activité
- **onStart()** : L'activité devient visible
- **onResume()** : L'utilisateur peut interagir
- **onPause()** : L'activité perd le focus
- **onStop()** : L'activité n'est plus visible
- **onDestroy()** : L'activité est détruite

Cycle de Vie d'une Activité Android

onCreate

Cette méthode est appelée lors de la création de l'activité. Elle initialise l'interface utilisateur et charge les ressources nécessaires.

Si cette activité est la première d'une application, cette méthode est donc appelée lorsque l'utilisateur exécute l'application.

onRestart

onRestart() est exécutée lorsque l'activité est redémarrée après un arrêt.`onStop`.

- Cette méthode est appelée après `onStop()` lorsque l'utilisateur revient à l'application après une interruption prolongée.
- Permet de réinitialiser certains éléments de l'interface utilisateur ou de relancer des processus arrêtés.

onStart

onStart() est exécutée après chaque appel à `onCreate()` ou `onRestart()`.

- Indique que l'activité devient visible à l'utilisateur.
- Prépare l'interface utilisateur et recharge les données sauvegardées si nécessaire.
- C'est ici que les animations ou les mises à jour peuvent être relancées.

onPause

onPause() est exécutée lorsqu'une activité passe en arrière-plan, mais n'est pas encore complètement arrêtée.

- Déclenchée dans les cas suivants :
 - L'utilisateur passe à une autre activité.
 - L'utilisateur ferme l'application (`finish()`).
 - Le système doit libérer de la mémoire.
- Toujours exécutée avant `onStop()`.
- Actions recommandées :
 - Sauvegarder les données critiques.
 - Gérer la déconnexion de la base de données.
 - Stopper les animations ou lectures multimédias.

Cycle de Vie d'une Activité Android

onStop

onStop() est exécutée lorsque l'activité n'est plus visible à l'écran. Elle précède systématiquement **onDestroy()** si l'activité est fermée.

- Exécutée avant chaque :
 - Mise en sommeil de l'application.
 - Appel de **onDestroy()** (fermeture définitive).
- Actions recommandées :
 - Libérer les ressources non essentielles (capteurs, connexions réseau).
 - Arrêter les mises à jour en arrière-plan.
 - Sauvegarder l'état de l'application.

onDestroy

L'activité est complètement détruite et toutes les ressources sont libérées.

Cycle de vie d'une activité

